Build a Large Language Model from Scratch

This thesis builds upon the approach described in [1], extending it by applying the methodologies to a different problem.

All code implementations developed for this report are available in the GitHub repository [2], specifically in the file chapter6.ipynb

The dataset used in this work was obtained from Kaggle [3] and is distributed under a CC0 (public domain) license, allowing unrestricted use for research and educational purposes.

REPORT 6

Fine-tuning for classification

Giorgi Gogsadze
Revaz Goguadze

Supervised by Prof. Walter Tichy

Structure

1. **Introduction**
2. **Dataset Preparation**
3. **Data Loader Construction**
4. **Model Adaptation for Classification**
5. **Evaluation Metrics**
6. **Model Fine-Tuning and Analysis**
7. **Discussion and Conclusion**

All code presented in this work is based on Chapter 6 of [1], with modifications and improvements introduced for the purposes of this study.

1. Introduction

This report aims to explain how a pretrained GPT language model was fine-tuned for supervised classification task. While chapter 6 of [1] guides to classification containing 2 classes, we chose to modify some of the codes and do it with any number of classes.

Our work is about emotion classification of short text messages. Unlike the original goal of GPT models, which are trained to predict the next token in a sequence, the objective here is to predict a class label from a fixed set: *positive*, *neutral*, and *negative*.

2. Dataset Preparation

The performance of our system is strongly dependent on the quality and structure of the dataset used for training. In the context of classification fine-tuning of a LLM, dataset preparation involves transforming raw text data into a balanced, numerical, and structured format suitable for model training.

Dataset Description

The dataset used in this work consists of **31,015** Twitter posts, each labeled with one of three sentiment categories: **positive**, **neutral** or **negative**. While each data instance contains two fields: **Label** (the sentiment class) and **Text** (the corresponding tweet).

	Label	**Text**
0	neutral	I`d have responded, if I were going
1	negative	Sooo SAD I will miss you here in San Diego!!!
2	negative	my boss is bullying me…
…	…	…
31013	positive	_sutra what is your next youtube video gonna b…
31014	positive	http://twitpic.com/4woj2 - omgssh ang cute n…

Class Distribution and Imbalance

An important aspect of the dataset is its class distribution, which is initially imbalanced:

- Neutral: 12,548
- Positive: 9,685
- Negative: 8,782

Class imbalance introduces a bias in the learning process. For example, predicting "neutral" for all inputs would already yield relatively high accuracy due to its dominance.

To address this issue all classes are randomly sampled to match the smallest class. The resulting balanced dataset contains:

- Positive: 8,782
- Neutral: 8,782
- Negative: 8,782

This process ensures that all classes contribute equally during training and the loss function does not favor any class.

Even though under sampling reduces dataset size, it is an effective and simple method for solving imbalance problem in classification tasks.

Label Encoding

Neural networks operate on numerical data. therefore, labels must be converted into integers. This is achieved using a factorization process, which assigns a unique integer to each class:

- positive → 0
- neutral → 1
- negative → 2

Dataset Shuffling and Splitting

To ensure that the model does not learn patterns based on how data is ordered, the dataset is **randomly shuffled**. Then we divided it into three subsets:

- **Training set (70%)** - Used to update model parameters
- **Validation set (10%)** - Used to monitor performance and guide training decisions
- **Test set (20%)** - Used for final, unbiased evaluation

This separation ensures that overfitting can be detected and the model is evaluated not only on training data, but also on unseen data.

3. Data Loader Construction

After preparing the dataset, the next step is to transform the data into a form that is the best for efficient training of the model. This involves tokenization (for which we use GPT-2 tokenizer), sequence normalization, and batching, all of which are handled through a custom dataset class and PyTorch data loaders.

Variable-Length Sequences and the Need for Padding

In pretraining the model uses fixed-length text chunks. However, in the classification dataset there are variable-length inputs, which is problem as batching requires that all sequences in a batch have the same length.

There are two main ways to address this problem:

1. **Truncation to shortest sequence**
    - computationally efficient, but leads to information loss
2. **Padding to longest sequence**
    - preserves all information, but increases computational cost

In this work, the second approach was selected, as the maximum input length in the dataset is sufficiently small to justify the additional computational cost.

The maximum sequence length was determined from the training dataset and turned out to be 70 tokens, which reflects the natural length of Twitter posts and is significantly smaller than the model's maximum context size (1024 tokens).

After tokenization all sequences are padded to a fixed length using the special token, that does not carry semantic meaning and acts as a placeholder:

$$< |\text{endoftext}| > \quad (\text{token ID } 50256)$$

So, a sequence shorter than the maximum length becomes:

$$[x_1, x_2, x_3] \rightarrow [x_1, x_2, x_3, 50256, 50256, \dots]$$

Exercise 6.1

An alternative configuration could be to pad all sequences to the full model context length (1024 tokens). However, after experiments, this resulted in a worse performance **(approx. 65% accuracy)**.

This can be explained by the fact that 70 tokens represent less than 7% of the context window and classification head operates on the final token, therefore there is a huge positional gap between real end of a tweet and the final token of the constructed input.

Thus, selecting an appropriate sequence length is critical for performance.

Custom Dataset Implementation

All above mentioned features has been united into a custom dataset class named MyDataset (chapter6.ipynb of [2]). Its responsibilities include:

1. **Loading data from CSV files**
2. **Filtering invalid entries** (removing missing or empty text)
3. **Tokenizing text**
4. **Determining sequence lengths**
5. **Applying truncation (if necessary)**
6. **Applying padding**

Each dataset item returns:

- input tensor → sequence of token IDs
- label → integer class index

Batch Construction with Data Loaders

PyTorch DataLoader objects are used to group data into batches, which enables efficient iteration by avoiding processing inputs one by one. In this project we decided to increase batch size to 32. This decision was motivated by the relatively small maximum input length (only 70 tokens) and the large dataset size (over 30k samples). Increasing the batch size to 32 resulted in more stable training and improved computational efficiency due to parallelism.

Each batch has the structure:

- Input tensor ([32,70])
- Target tensor ([32])

4. Model Adaptation for Classification

Like last time, the model will be loaded with pretrained GPT model weights before we start fine-tuning. however, GPT is originally designed for language modeling, where the objective is to predict the next token in a sequence. To apply this model to a classification task, its architecture and training objective must be modified.

From Language Modeling to Classification

In its original form, the GPT model outputs a tensor of shape:

$$(\text{batch size}, \text{sequence length}, \text{vocabulary size})$$

For GPT-2, the vocabulary size is 50,257. However, for classification tasks we want to assign a single class label to the entire input sequence. Thus, the model must be adapted to map a sequence of tokens to a **fixed number of classes**, which in our case is 3.

The key architectural modification is the replacement of the original output layer, the one which produces logits, with a **classification head**. So instead of mapping embedding dimension into 50,257 logits, it will map to only 3.

Then for a given input, the model will output $[z_0, z_1, z_2]$, where each value corresponds to a class and the one with highest value will be the predicted class.

Classification head

A classification head is a small neural network layer added on top of a pretrained model to adapt it for a specific task, such as classification.

In the original GPT model, the final layer (called the *output layer*) is designed for language modeling, where each output corresponds to a token in the vocabulary. For classification tasks, this behavior is not needed. Instead, we replace this output layer with a classification head.

- The classification head takes the same 768-dimensional hidden representation.
- However, instead of predicting thousands of tokens, it outputs a small number of values equal to the number of classes.

Figure 1 illustrates this change:

- **Before:** 768 → 50,257 (next-token prediction)
- **After:** 768 → *n* (class prediction, where *n* is the number of classes)

This modification allows the model to reuse its learned language representations while adapting it to a new task.

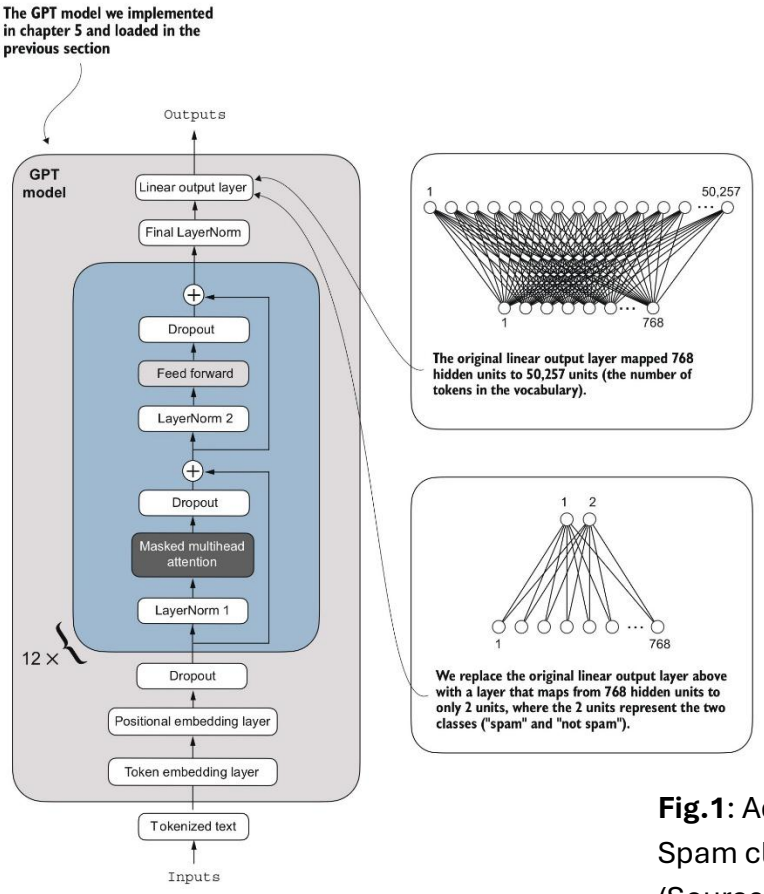


Fig.1: Adapting a GPT model for Spam classification.
(Source: [1], p. 184)

Freezing Model Parameters

For this work, we used GPT-2 Medium (355M parameters), which is large model with improved ability to capture nuanced patterns. After initializing the model with pretrained weights, we prepare it for further training, specifically classification fine-tuning. However, not all layers in the model need to be trained. The lower layers typically capture general linguistic patterns, which should be preserved, while the higher layers learn task-specific features. Thus, we freeze lower layers, which:

- reduces computational cost
- preserves useful pretrained knowledge
- avoids overfitting

As a result, only a subset of the model is trained, specifically:

- Final transformer block
- Final layer normalization
- Classification head

Exercise 6.2

An alternative approach involves unfreezing all parameters:

```
for param in model.parameters():
    param.requires_grad = True
```

This resulted in slightly improved accuracy to approximately **80%**, but at the same time training time increased to approximately **4 hours.**

This tells us that the full fine-tuning maybe enables the model to better align with task-specific patterns, but it is very expensive computationally.

Selecting the Relevant Output Token

Note that the model produces an output for each token in the input sequence. For an input of length 70, this results in 70 output vectors, whereas classification requires only a single prediction per input. Due to the attention mechanism masking future tokens, only the final token has access to the full input context. Therefore, the output corresponding to the last token is used as the most appropriate representation for classification.

Exercise 6.3

We conducted an experiment in which the indexing was changed from (slice(None), -1, slice(None)) to (slice(None), 0, slice(None)), meaning that the first token representation was used as an output instead of the last. This modification resulted in an accuracy of approximately 70%, which is lower than the performance obtained when using the last token. This outcome is expected, as the first token has access to less contextual information than the final one.

5. Implement Evaluation Metrics

After adapting the model architecture for classification, the next step is to implement classification loss and accuracy functions, so we can evaluate model performance during training and testing.

Accuracy function

After the forward pass, the model produces logits of the form:

$$\text{logits} = \text{model}(\text{x})[:, -1, :]$$

As mentioned above, these logits correspond to the **last token output**. Finally, the output logit will look something like this: $[-3.6345, 1.1317, -0.0587]$ (chapter6.ipynb [2]).

To obtain a prediction from this tensor, the index of the maximum logit is selected using argmax function. After getting predictions for all inputs, accuracy measures the proportion of correctly predicted labels:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

Cross-Entropy Loss

We are using same loss function calculating strategy, that we defined for pretraining:

$\mathcal{L} = -\log(p_y)$ where p_y is the predicted probability of the true class.

This means:

- If the model assigns high probability to the correct class → low loss
- If the model is uncertain → higher loss
- If the model is confidently wrong → very high loss

Thus, minimizing cross-entropy encourages correct predictions.

Implementation Details

Although the loss is defined using softmax probabilities, in practice the function:

```
torch.nn.functional.cross_entropy(logits, target_batch)
```

operates directly on logits and internally applies the softmax transformation. Therefore, it is not necessary to explicitly compute probabilities during training.

Similarly, class predictions can be obtained using the **argmax** function on logits, without applying softmax, since both yield the same result.

As in the pretraining stage, the model is optimized using the **AdamW optimizer**, which is widely used for transformer-based architectures due to its stability and effective handling of weight decay.

Baseline Performance Before Fine-Tuning

Before training, the model achieves:

- Training accuracy: 34.38%
- Validation accuracy: 35.94%
- Test accuracy: 35.31%

For a 3-class problem:

$$\text{Random accuracy} = \frac{1}{3} \approx 33.3\backslash\%$$

Thus, the pretrained model performs randomly. This shows that pretrained GPT models do not inherently perform classification and task-specific fine-tuning is necessary

6. Model Fine-Tuning and Analysis

The training algorithm necessary for fine-tuning is basically same as it was for pretraining, explained deeply in report 5. The modification is that instead of generating sample text during training, the model is evaluated using classification accuracy after each epoch. which transits training from a language modeling task to a classification task.

Early Training Phase (Epochs 1–2)

- Stabilization around `Train loss 1.096, Val loss 1.092`
- Low accuracy `Training accuracy: 40.62% | Validation accuracy: 31.25%`

The model initially behaves like a **random classifier**, assigning equal probability to all classes. This is expected because the dataset is large and two epochs aren't enough for meaningful learning. In contrast, the behavior observed in Chapter 6 of [1] shows that smaller datasets can lead to faster improvements, with accuracy reaching around 70% after the first epoch.

Learning Phase Transition (Epoch 3)

From epoch 3 onward:

- Training as well as validation loss decreases rapidly (from ~1.09 to ~0.85 and below)
- Accuracy increases significantly (~70%)

This marks a **transition point** where the model trainable layers began to learn meaningful, classifier-specific patterns. This delayed improvement is also typical when most layers are frozen, as in our case.

Late Training Phase (Epochs 4–5)

```
Ep 4 (Step 001950): Train loss 0.695, Val loss 0.690
...
Training accuracy: 76.25% | Validation accuracy: 75.00%
...
Ep 5 (Step 002550): Train loss 0.659, Val loss 0.614
```

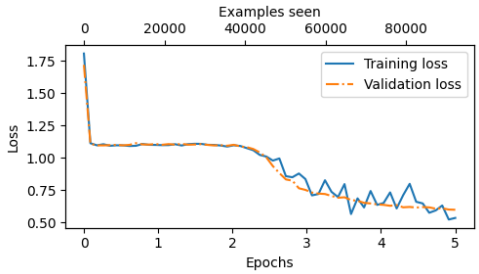
In final epochs:

- Both training and validation losses continue to decrease and remain close, indicating a good generalization and the absence of significant overfitting.
- Accuracy stabilizes at approximately 75%, suggesting convergence of the model.

Final Results

Training took 22.70 minutes, which is considered as very efficient, taking the dataset and model sizes into consideration. It also showed a solid results.

The loss plot:

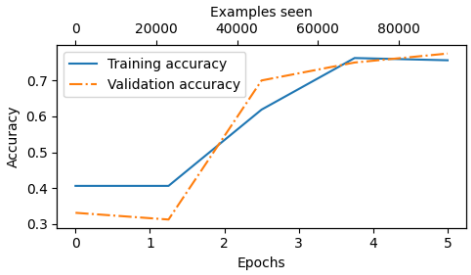


The sharp drop in the first few hundred examples are typical in classification fine-tuning, as classification head weights start shaping here. At this point guesses are purely random, and taking into consideration we have 3 classes, initial convergence to 1.1 is expected:

$$\mathcal{L} = -\log\left(\frac{1}{3}\right) \approx 1.098$$

However, in first and second epochs loss plateaus, staying close to the random-guessing baseline. Meaningful learning only begins in epoch 3 and by epoch 5 converges to 0.5. Overall, the curves look healthy: both accuracies rise together and plateau. No clear sign of overfitting.

The accuracy plot:



This plot shows a very successful fine-tuning run for most practical purposes.

- Accuracy keeps improving until late epochs and then starting converging.
- Very slight divergence between train and validation metrics means almost no overfitting.
- validation metrics being slightly better than training metrics is a good sign of generalization. It's quite common in fine-tuning.

Final results showed:

```
Training accuracy: 76.04%
Validation accuracy: 74.83%
Test accuracy: 74.54%
```

These numbers show strong consistency across the three splits, with only a modest ~1.5% gap between training and test accuracy, Indicating a good generalization.

Achieving approximately **75%** accuracy on this task is a reasonable outcome. The dataset was large, noisy, and contained many ambiguous or subjective sentences, making perfect classification challenging. In this context, the model demonstrates solid performance in capturing sentiment patterns, which will be furthered shown by the qualitative testing.

Qualitative Evaluation

In addition to quantitative metrics, a qualitative analysis was conducted by testing the model on several unseen sentences. All test cases are available for further experimentation in the "Test Classifier Model.ipynb" notebook in the GitHub repository [2].

The model performed well on straightforward examples:

Sentence	**Model Guess**
I did amazing in the exam today. Finally, I'll get good grades.	Positive
Today I got so tired, whole day I'm working like a dog. Sometimes I just want to sleep and do nothing.	Negative
The parliament of Georgia elected new Prime minister today.	Neutral
today was my first day at the university. It wasn't really good!	Negative
I can't say this wasn't one of the best experiences I've had.	Positive

This shows that:

- clearly emotional statements are correctly classified
- neutral factual statements are correctly identified

However, if we give the model slightly complex sentence, it shows its limitations:

```
text = ("I'm not unhappy with the results. quite the opposite.")
print(classify_review(
      text, model, tokenizer, device, max_length=train_dataset.max_length
))
```

The model incorrectly classified this sentence as **negative**, even though the intended meaning is clearly **positive** (a double negation expressing satisfaction).

This example also highlights a common issue that is frequently faced in sentiment analysis, where the interpretation of linguistic aspects, such as negation, sarcasm, and even underlying sentiment, is a challenge for the model. These mistakes highlight that, even though the fine-tuned GPT-2 model has achieved strong overall performance, there is room for improvement.

7. Discussion and Conclusion

This report successfully demonstrates how a pretrained GPT-2 Medium model, that already embeds semantic information of a language, is a great tool and can be fine-tuned for a classification task. By modifying the original approach from Chapter 6 of [1], we built an architecture that supports an arbitrary number of classes and extended the methodology to a more realistic multi-class emotion detection problem.

One of the key observations from this study is the trade-off between efficiency and performance. We've seen that full fine-tuning yields slightly better performance at the expense of increasing the computational cost. Therefore, efficiency of the partial fine-tuning turned out to be superior.

In conclusion, the implemented pipeline provides a solid foundation for similar projects and clearly illustrates the power of fine-tuning in natural language processing. All code and experiments are openly available in the accompanying GitHub repository for reproducibility and further extension.

References

[1] Raschka, S. (2024). *Build a Large Language Model (From Scratch)*. Shelter Island, NY: Manning Publications.

[2] Gogsadze, G., Goguadze, R. (2026). *LLM From Scratch Code Repository*. Available at: https://github.com/GiorgiGogsadze/LLM-From-Scratch-Thesis

[3] Shrivastava, A. (2020). *Sentiment Analysis Dataset*. Kaggle. Available at: https://www.kaggle.com/datasets/abhi8923shriv/sentiment-analysis-dataset